

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-24T10:05:43.7990501Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260924-100543800.prof
Profil mémoire	memory-20260924-100543800.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	6.27 ms	6265305.2	0.4	0

Détail hyperfine

Métrique	Valeur
Moyenne	4.9277 s
Écart-type	0.2214 s
Médiane	5.0238 s
Temps utilisateur	17.8728 s
Temps système	1.0763 s
Min	4.5454 s
Max	5.0952 s
Runs mesurés	5
Coefficient de variation	4.49%
Pic WorkingSet commande Go	27.01 MiB
Pic mémoire privée commande Go	60.77 MiB

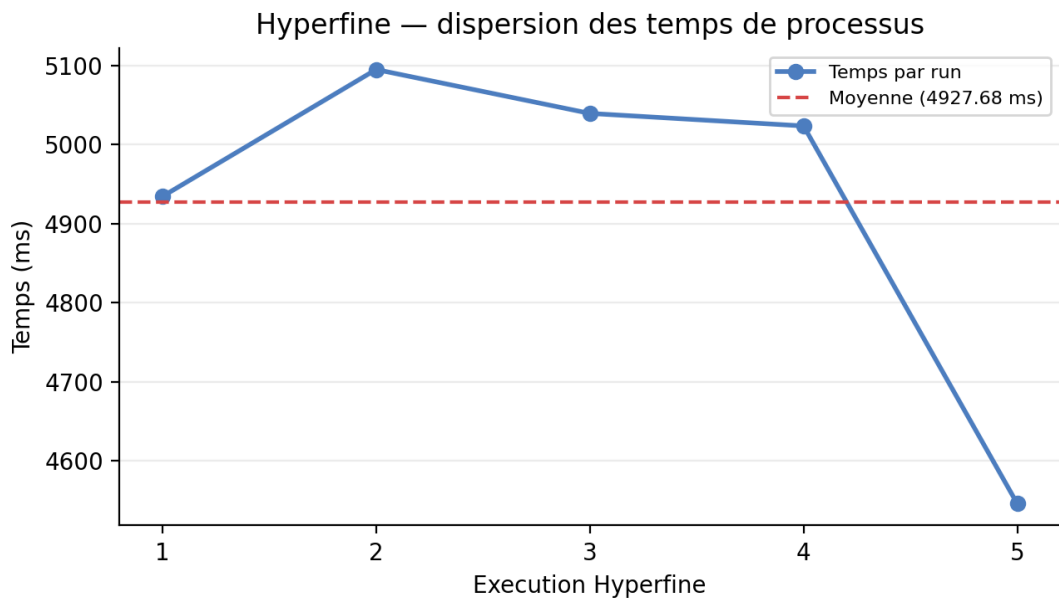


Figure — Temps de chaque exécution Hyperfine et moyenne.

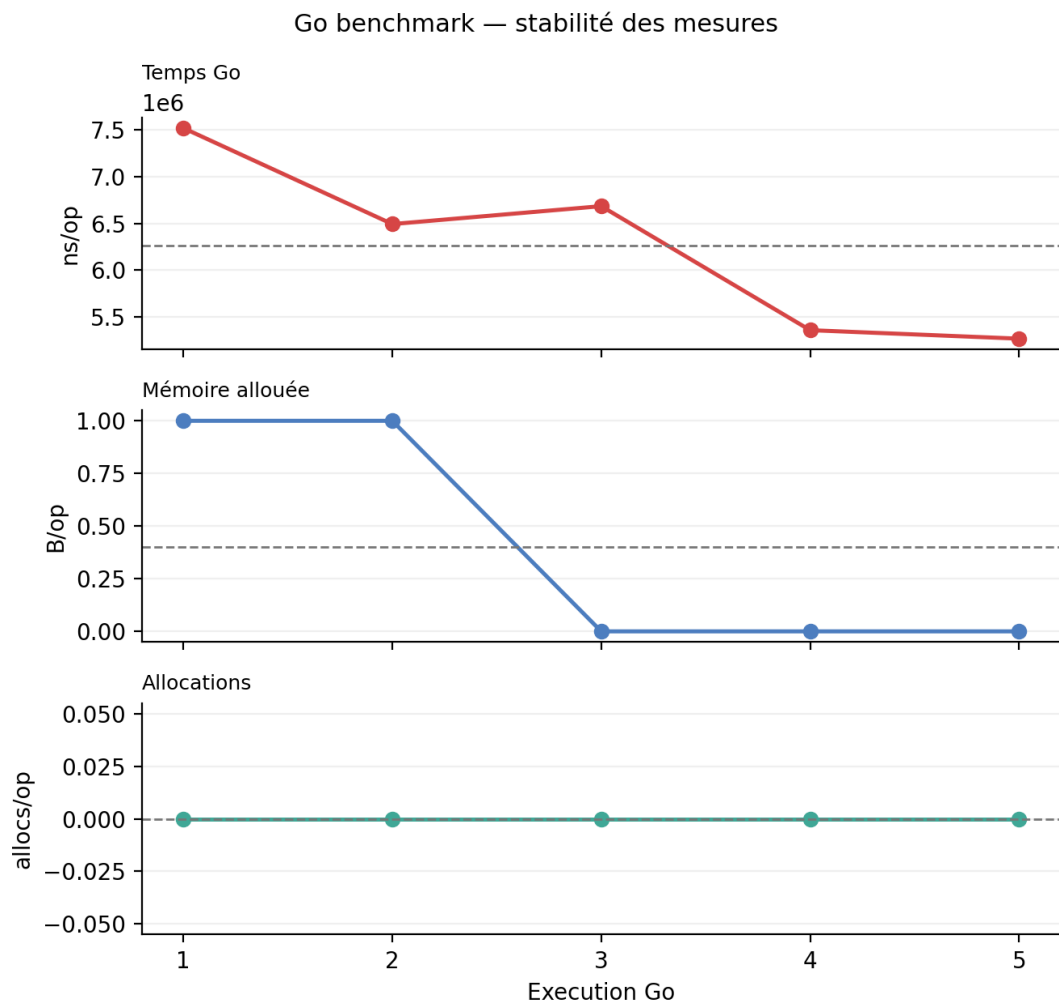


Figure — Dispersion des mesures Go : temps, mémoire allouée et allocations.

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	17.86s
Mémoire allouée (pprof)	69.46MB
Heap vivant (pprof)	6160.12kB
RAM physique totale	32061 MiB
RAM physique disponible	14984 MiB
RAM physique utilisée	17077 MiB
Mémoire par opération	0.4 B/op

Garbage collector Go

Cycles GC	65
Temps GC cumulé	64.470 ms
Plus longue pause	5.000 ms
Temps CPU GC cumulé	360.990 ms
Objectif mémoire maximal	97.00 MB

La RAM physique est l'état du système au lancement du benchmark. La mémoire pprof mesure le heap Go échantillonné (alloué ou encore vivant), pas toute la mémoire RSS du processus. Hyperfine peut fournir le RSS quand le backend système le supporte ; une valeur nulle est donc signalée comme indisponible.

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **17.86s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).infectionCandidate	13.44s	75.25%	15.85s	88.75%
jeu-de-la-vie/game.(*Board).index (inline)	2.15s	12.04%	2.16s	12.09%
jeu-de-la-vie/game.(*Board).Step	0.98s	5.49%	1.63s	9.13%
math/rand.(*Rand).Float64 (inline)	0.31s	1.74%	0.60s	3.36%
jeu-de-la-vie/game.startCandidatePool.func1	0.26s	1.46%	16.15s	90.43%
jeu-de-la-vie/game.squaredDistance (inline)	0.17s	0.95%	0.17s	0.95%
math/rand.(*rngSource).Uint64 (inline)	0.16s	0.90%	0.16s	0.90%
runtime.asyncPreempt	0.11s	0.62%	0.11s	0.62%

Component	Temps CPU cumulé (s)	Pourcentage
actionCandidateCount	13.44s	75.3%
index (inline)	2.15s	12.0%
Step	0.98s	5.5%
Float64 (inline)	0.31s	1.7%
reste	0.98s	5.5%

File: game.test.exe
 Build ID: C:\Users\saiou\AppData\Local\Temp\go-build1733537600\b001\game.test.exe2026-09-24 12:05:44.7239969 +0200 CEST
 Type: cpu
 Time: 2026-09-24 12:05:45 CEST
 Duration: 3.33s, Total samples = 17.86s (536.24%)
 Showing nodes accounting for 17.71s, 99.16% of 17.86s total
 Dropped 77 nodes (cum <= 0.09s)

See <https://git.io/JfYMW> for how to read the graph

```

graph TD
    Root["game startCandidatePool  
func1  
0.26s (1.46%)  
of 16.15s (90.43%)"]
    Infection["game (*Board)  
infectionCandidateCounts  
13.44s (75.25%)  
of 15.85s (88.75%)"]
    Index["game (*Board)  
index  
2.15s (12.04%)  
of 2.16s (12.09%)"]
    Benchmark["game BenchmarkTick  
0.16s (0.89%)  
of 1.64s (9.26%)"]
    Step["game (*Board)  
Step  
0.98s (5.49%)  
of 1.63s (9.13%)"]
    Rand64["rand (*Rand)  
Float64  
0.31s (1.74%)  
of 0.60s (3.36%)"]
    RandInt3["rand (*Rand)  
Int32  
0.03s (0.45%)  
of 0.29s (1.62%)"]
    RandInt3Source["rand (*randSource)  
Int32  
0.03s (0.20%)  
of 0.21s (1.18%)"]
    RandInt3Sink["rand (*randSource)  
Int32  
0.16s (0.99%)  
of 0.16s (0.99%)"]
    TimeAsyncPreempt["runtime.asyncPreempt  
0.11s (0.62%)"]
    GameSquareDistance["game.squareDistance  
0.17s (0.92%)"]

    Root -- 15.85s --> Infection
    Infection -- 0.02s --> TimeAsyncPreempt
    Infection -- 0.08s --> GameSquareDistance
    Infection -- 0.17s (inline) --> Index
    Index -- 2.16s (inline) --> Benchmark
    Benchmark -- 1.63s --> Step
    Step -.- 0.60s (inline) --> Rand64
    Rand64 -- 0.29s (inline) --> RandInt3
    RandInt3 -- 0.21s --> RandInt3Source
    RandInt3Source -- 0.16s (inline) --> RandInt3Sink
  
```

Figure — Graphe d'appels CPU (go tool pprof -png).

2.1 Diagnostic automatique

La fonction dominante de ce run est `jeu-de-la-vie/game.(*Board).infectionCandidateCounts`, responsable de **75.2%** du temps CPU total (flat), pour **88.8%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

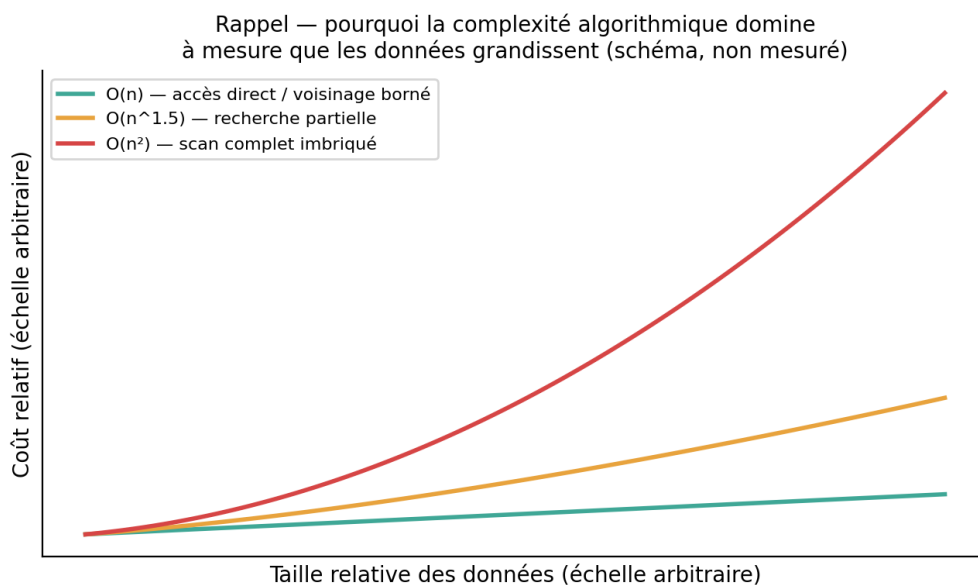


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **69.46MB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.NewBoard	46.26MB	66.60%	46.26MB	66.60%
jeu-de-la-vie/game.RandomBoard	13.81MB	19.88%	60.07MB	86.47%
runtime.mallocgc	3.01MB	4.33%	3.01MB	4.33%
time.initLocalFromTZI	1MB	1.44%	2.50MB	3.60%
compress/flate.NewWriter (inline)	0.88MB	1.27%	0.88MB	1.27%
time.map.init.1	0.51MB	0.73%	0.51MB	0.73%
encoding/base64.init	0.50MB	0.72%	0.50MB	0.72%
jeu-de-la-vie/game.startCandidatePool.func1	0.50MB	0.72%	0.50MB	0.72%

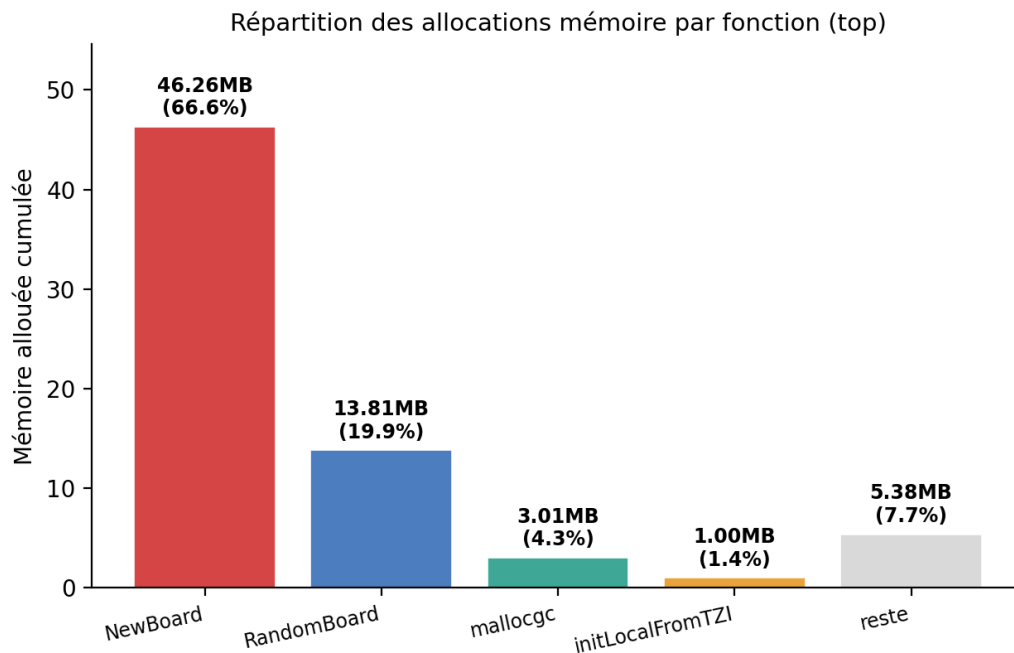


Figure — Répartition des allocations mémoire par fonction.

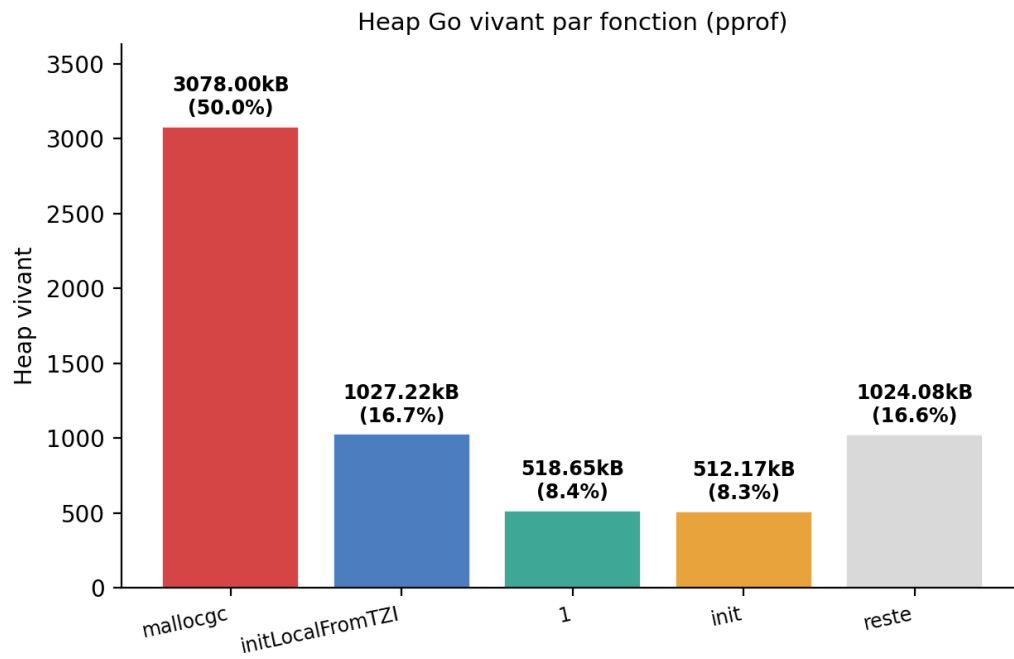


Figure — Heap Go encore vivat par fonction (pprof -inuse_space).

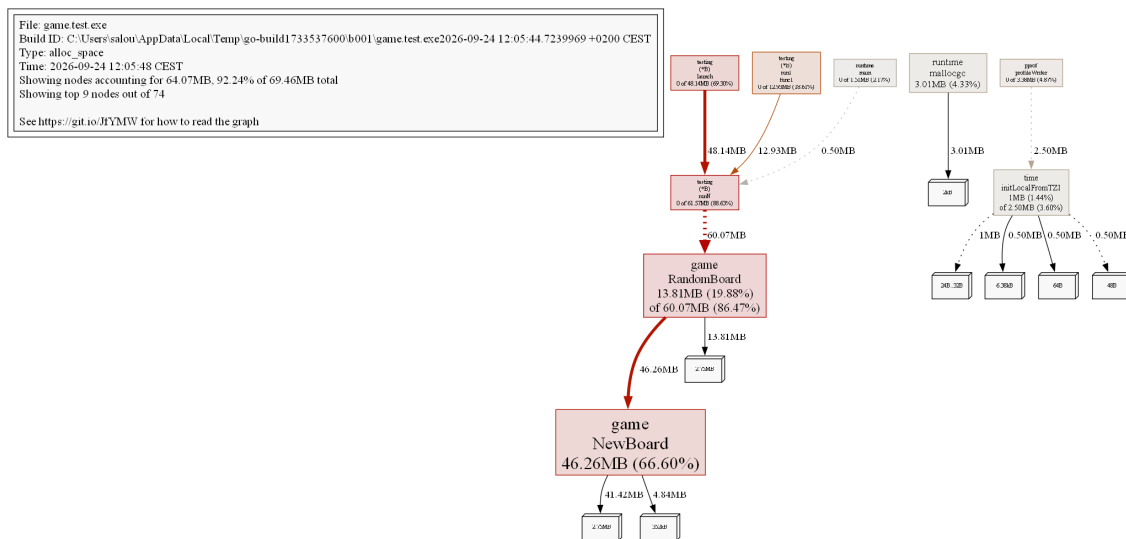


Figure — Graphe d'appels mémoire (go tool pprof -png).

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- La trace GC détaillée est enregistrée dans le fichier `gc-\*.log` et résumée dans le PDF.
- Les métriques I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-24 12:06:34 par generate_benchmark_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.